

CTE Example

SQL School Notes



1

Readability Win

We want to find courses whose fee is greater than the average course fee. A CTE makes this logic easier to understand because the average is named first, then used later.

2

CTE Name

AverageCourseFee is a self-documenting name. It tells us exactly what the temporary result set contains.

Good names = readable SQL

3

Example Query

```
WITH AverageCourseFee AS (  
  SELECT AVG(fee) AS avg_fee  
  FROM courses  
)  
SELECT title, fee  
FROM courses  
WHERE fee > (SELECT avg_fee FROM AverageCourseFee);
```



4

Why This Is Better

- Logic is split into steps
- Average is calculated once in a named CTE
- The query reads top to bottom
- Easier to explain and maintain



In Simple Words

First calculate the average fee, then return only the courses whose fee is higher than that average.



CTE Syntax

SQL School Notes



1

Syntax

A CTE starts with the **WITH** keyword. Inside parentheses, you write a normal SQL query. After that, the main query can use the CTE name just like a table.

2

A. Basic Single CTE

```
WITH temporary_table_name AS (  
  SELECT column_name  
  FROM table_name  
  WHERE condition  
)  
SELECT *  
FROM temporary_table_name;
```

Define first
→ use later



3

B. Multiple CTEs (Comma Trick)

```
WITH first_cte AS (  
  SELECT ...  
) , second_cte AS (  
  SELECT ...  
)  
SELECT *  
FROM first_cte  
INNER JOIN second_cte ON ...;
```

Separate
multiple CTEs
with commas.



Remember

- WITH comes first
- Each CTE gets a name

- AS (...) contains the query
- Main query uses the CTE like a table



★ CTE Best Practices ★

SQL School Notes • Day 20 Focus

1 Only One WITH Keyword

A huge beginner mistake is writing:
`WITH cte1 AS (...), WITH cte2 AS (...)`

→ You write WITH only one time at the very top.

After that, separate additional CTEs with commas.

```
WITH cte1 AS (  
  SELECT ...  
)  
,  
  cte2 AS (  
  SELECT ...  
)  
SELECT *  
FROM cte1  
JOIN cte2 ON ...;
```

WITH once ✓
Commas after that

2 Scope / Lifespan

CTEs are strictly temporary.

They exist only while the query is running.

As soon as the query finishes, the CTE vanishes from memory.

⚠ It does NOT permanently create a table in your database.

Temporary only •
not a real permanent table

3 CTEs vs Subqueries (Performance)

In modern PostgreSQL, CTEs and Subqueries usually have the same performance.

→ The choice is mostly about human readability.

★ If another developer needs more than 10 seconds to understand your subquery, rewrite it as a CTE!

CTE


Subquery
(SELECT ...
FROM ...
...)

99%
readability



Quick Tips

① Use WITH only once

② Separate multiple CTEs with commas

③ CTEs vanish after the query ends

④ Choose CTEs for readability



CTE Homework

SQL School Notes

1 The Temporary Roster

Write a CTE named AdminsOnly that selects all users where the role is 'admin'. Then write an outer query that selects everything from your new AdminsOnly CTE.



2 The Enrollment Math

Create a CTE called CourseCounts that groups by course_id and counts how many students are in each course from the enrollments table. In your main outer query, select from that CTE and use a WHERE clause to show only courses with more than 1 student.



3 The Clean Join

Write two CTEs separated by a comma. Name the first one CompletedPayments and filter for status = 'completed'. Name the second one Professors and filter for role = 'professor'. Then write an outer query that selects from both CTEs just to practice chaining multiple CTEs.



Homework Rules

- No solutions on this page
- Write the SQL yourself
- Focus on clean CTE syntax



CTE (Common Table Expression)

SQL School Notes

Exists only for that query ✓

1 Definition

A CTE (Common Table Expression) is a named temporary result set defined inside a SQL statement. You can think of it like a temporary, virtual table that exists only while that statement is running. It helps break complex logic into smaller, easier-to-read steps.

2 WITH Keyword

WITH is the keyword used to declare a CTE. It appears before the main SELECT, INSERT, UPDATE, or DELETE statement and gives a name to the temporary result set.

➡ Always starts at the top of the query.



3 Readability (Top-to-Bottom)

CTEs improve readability because you define the logic first and use it later. Unlike many subqueries, which are often read from the inside out, CTEs let you read SQL from top to bottom in a more natural flow.

Feels more like using clean variables in JavaScript. ♥



Quick Points

- Improves readability
- Breaks logic into steps
- Reusable in the same statement
- Great for complex queries

